

MédiHelm

Analyse Complète du Codebase vs Documentation

Projet : MédiHelm - Infrastructure Pharmaceutique Numérique du Bénin

Dépôt : <https://github.com/SenaDev007/MediHelm.git>

Éditeur : YEHI OR Tech - Dawes, Fondateur & Lead Developer

Date d'analyse : 15 juin 2026

Ce rapport présente une analyse exhaustive du codebase actuel du projet MédiHelm, comparé point par point avec la documentation officielle (CONTEXT.md, medihelm.md, medihelm.cursorrules, Cahier des Charges v2.0, Spécifications v2.0). Chaque couche du projet est examinée : architecture, base de données, API, frontend, sécurité, et infrastructure.

Sommaire

- 1. Vue d'ensemble du projet
- 2. Architecture : Documentation vs Réalité
- 3. Analyse du Schéma Prisma (Base de données)
- 4. Analyse des Routes API
- 5. Analyse du Frontend (Pages & Composants)
- 6. Sécurité et Multitenancy
- 7. Stack Technique : Écarts Documentés vs Réels
- 8. Modules : État d'Avancement
- 9. Risques Critiques Identifiés
- 10. Points Forts du Codebase
- 11. Recommandations Prioritaires

1. Vue d'ensemble du projet

MédiHelm est une plateforme SaaS multitenant conçue comme l'infrastructure pharmaceutique numérique du Bénin. Elle vise à connecter les pharmacies d'officine, les patients, les grossistes répartisseurs (UbiPharm, Promopharma) et les institutions de tutelle (DPMED, SoBAPS, ABRP) sur une plateforme unique. Le projet se distingue des simples logiciels de gestion de pharmacie par ses modules institutionnels de pharmacovigilance, de diffusion d'alertes DPMED et de traçabilité de la chaîne pharmaceutique.

1.1 Chiffres clés du codebase

Métrique	Valeur
Fichiers totaux	922
Modèles Prisma	75
Enums Prisma	31
Routes API	112 fichiers route.ts
Pages frontend	60+ pages avec contenu réel
Composants UI (shadcn)	45 composants
Composants métier	36 composants (pro, patient, institution, grossiste)
Hooks React	5 (use-upload, use-pwa, use-mobile, use-toast, use-locale)
Lib utilitaires	15 modules (auth, rbac, pdf, excel, fedapay, etc.)

1.2 Vision documentée

La documentation définit 19 modules fonctionnels (M01 à M19), un système RBAC à 12 rôles, 4 plans tarifaires (SEED, GROW, LEAD, NETWORK), et 6 partenaires institutionnels. Le tout devait être implémenté dans une architecture monorepo avec un backend NestJS distinct et un frontend Next.js. L'analyse qui suit révèle que la réalité du codebase diffère significativement de cette vision documentée, tout en démontrant un niveau d'implémentation fonctionnel remarquable.

2. Architecture : Documentation vs Réalité

C'est l'écart le plus significatif entre la documentation et le codebase réel. La documentation (CONTEXT.md, Cahier des Charges, Monorepo_Portails) décrit une architecture monorepo avec séparation backend/frontend, tandis que le code est une application Next.js unifiée.

2.1 Architecture documentée

Composant	Documentation	Réalité
Structure	Monorepo (apps/api + apps/web + packages/)	Application Next.js unique (src/)
Backend	NestJS + Fastify (apps/api/)	AUCUN backend NestJS - Route Handlers Next.js
Frontend	Next.js 14 App Router (apps/web/)	Next.js 16 App Router (src/app/)
Packages partagés	packages/shared-types, ui, utils	AUCUN - types inline dans chaque page
WebSocket	NestJS @WebSocketGateway()	SSE via /api/notifications/stream (pas de WebSocket)
Séparation des portails	Route groups (pro), (patient), (network), (institutionnel)	Route groups /pro/, /patient/, /institutions/, /grossistes/

2.2 Impact architectural

L'absence de backend NestJS signifie que toute la logique métier réside dans les Route Handlers Next.js, ce qui fonctionne pour un MVP mais pose des problèmes de scalabilité et de séparation des responsabilités. Les 112 routes API sont des fonctions serverless qui ne peuvent pas maintenir de connexions persistantes (WebSocket), de files d'attente BullMQ, ou de cache Redis entre les requêtes. L'absence de monorepo élimine aussi le partage de types entre frontend et backend, augmentant le risque d'incohérence.

Le choix de Next.js 16 (au lieu du 14 documenté) avec React 19 apporte des fonctionnalités modernes mais n'était pas prévu dans les spécifications. Le groupe de routes (network) documenté est absent, remplacé par /pro/reseau.

3. Analyse du Schéma Prisma (Base de données)

Le schéma Prisma est la couche la plus aboutie du projet, avec 75 modèles couvrant l'ensemble des 19 modules documentés. Cependant, des divergences notables existent avec les spécifications.

3.1 Divergences critiques entre schéma et documentation

Modèle	Champ documenté	État réel	Impact
Vente	reference (@unique)	ABSENT	CRITIQUE - Sync offline impossible sans référence unique
Vente	synchedAt (DateTime?)	ABSENT	CRITIQUE - Impossible de distinguer ventes online/offline

Modèle	Champ documenté	État réel	Impact
Vente	montantAssur	ABSENT	MOYEN - Pas de montant assurance sur la vente
Vente	commandePatId	ABSENT	MOYEN - Pas de lien commande patient vers vente
Pharmacie	slug (@unique)	ABSENT	FAIBLE - Pas d'URL conviviale
Pharmacie	modeGardeActif	ABSENT	FAIBLE - Pas de flag garde active
Pharmacie	planExpireAt	ABSENT	MOYEN - Pas de suivi expiration plan
Utilisateur	supabaseUid (@unique)	ABSENT	MOYEN - Auth NextAuth au lieu de Supabase
Medicament	surOrdonnance	ABSENT	MOYEN - Pas de flag prescription requise
Medicament	stockSecurite	ABSENT	FAIBLE - Seulement stockMin existe
AnalyticsReport	Modèle complet (scoreGlobal, alertes, etc.)	RapportAnalytics générique (Json)	MOYEN - Perte de typage sur les analytics
DomaineIA	Enum (9 valeurs)	ABSENT - domaine: String	MOYEN - Pas de validation des domaines

3.2 Divergences d'enums

Enum documenté	Valeurs documentées	État réel
PlanType	STEM / GROW / LEAD / NETWORK	PlanTarifaire : SEED / GROW / LEAD / NETWORK (STEM renommé SEED)
RoleType	Enum inline (12 valeurs)	Tables Role + Permission + RolePermission (RBAC relationnel)
StatutVente	VALIDEE / ANNULEE / AVOIR / PARTIELLEMENT_PAYEE	EN_COURS / VALIDEE / ANNULEE / REMBOURSEE (AVOIR absent)
ModePaiement	ESPECES / WAVE / MTN_MONEY / MOOV_MONEY / CARTE / ASSURANCE / CREDIT	ESPECES / CARTE / MOBILE_MONEY / WAVE / MTN_MONEY / MOOV_MONEY / TIERS_PAYANT (ASSURANCE et CREDIT absents)
StatutCmdPat	RECUE / EN_PREPARATION / PRETE / RECUPEREE / ANNULEE	PENDING / CONFIRMED / PREPARING / READY / PICKED_UP / CANCELLED (en anglais !)

3.3 Éléments positifs du schéma

Le schéma Prisma est remarquablement complet et couvre tous les 19 modules avec des modèles détaillés. Le système RBAC relationnel (Role/Permission/RolePermission) est plus sophistiqué que le simple enum documenté, permettant une gestion fine des permissions par module et action. Le modèle AlerteDPMED est plus riche que prévu avec des compteurs de diffusion et

d'acquiescement. Le seed script (1019 lignes) est de qualité production avec des données réelles de Parakou et des coordonnées OpenStreetMap.

3.4 Problèmes de migration

AUCUNE migration Prisma n'existe. Le projet utilise prisma db push, ce qui est acceptable en développement mais inapproprié pour la production car il ne permet pas de rollback. Le script scripts/update-pharmacies-osm.ts contient des identifiants de base de données Neon en clair dans le code source, ce qui constitue une faille de sécurité.

4. Analyse des Routes API

Le projet compte 112 fichiers route.ts couvrant l'ensemble des domaines métier. Toutes les routes ont une logique Prisma réelle avec gestion d'erreur try/catch. Cependant, des lacunes majeures en sécurité et en conformité avec la documentation sont identifiées.

4.1 Endpoints documentés vs implémentés

Endpoint documenté	Statut	Notes
POST /auth/login	IMPLÉMENTÉ	Via NextAuth /api/auth/[...nextauth]
POST /auth/refresh	ABSENT	NextAuth gère le refresh en interne mais pas d'endpoint explicite
GET /medicaments	IMPLÉMENTÉ	/api/medicaments avec filtre pharmacieId
POST /ventes	IMPLÉMENTÉ	/api/ventes
POST /ventes/sync	ABSENT	CRITIQUE - Pas de synchronisation offline des ventes
GET /stock/alertes	IMPLÉMENTÉ	/api/stocks/alertes
POST /grossistes/:id/commandes	IMPLÉMENTÉ	/api/grossistes/[id]/commandes
POST /sobaps/receptions	IMPLÉMENTÉ	/api/sobaps/receptions
POST /webhooks/dpmed	IMPLÉMENTÉ	Avec validation HMAC-SHA256
POST /webhooks/ubipharm	IMPLÉMENTÉ	Avec validation HMAC-SHA256
POST /webhooks/promopharma	IMPLÉMENTÉ	Avec validation HMAC-SHA256
POST /alertes/dpmed/:id/acquitter	IMPLÉMENTÉ	/api/alertes/dpmed/[id]/acquitter
POST /analytics/relancer	ABSENT	Pas d'endpoint de relance analytics avec cooldown
GET /public/medicaments/search	IMPLÉMENTÉ	/api/patient/recherche (public)
GET /public/gardes	IMPLÉMENTÉ	Via /api/pharmacies?garde=semaine
POST /public/commandes	IMPLÉMENTÉ	/api/patient/commandes (public)

4.2 Sécurité des routes API - Problème critique

Seulement 7 routes sur 112 appliquent une vérification d'authentification via `requireAuth()` ou `getAuthUser()`. Les routes utilisant `requireAuth` sont : `/api/stupefiants`, `/api/ticket`, `/api/exports/stock`, `/api/exports/ventes`, `/api/exports/patients`, et `/api/notifications/stream`. Toutes les autres routes (105 sur 112) n'ont aucune vérification d'authentification dans le handler, bien que le middleware `NextAuth` vérifie la présence du cookie de session.

L'infrastructure RBAC est entièrement construite (`rbac.ts` avec 11 rôles, 19 modules, permissions `read/write/delete`) mais n'est quasiment pas utilisée. Le helper `requirePharmacieAccess()` existe dans `api-auth.ts` mais n'est jamais appelé par aucune route. Cela signifie qu'un utilisateur authentifié pourrait potentiellement accéder aux données d'une autre pharmacie en modifiant le paramètre `pharmacieId`.

4.3 Validation des entrées

La majorité des routes API ne validant pas les entrées côté serveur. Les données du body sont souvent passées directement aux requêtes Prisma sans schéma de validation (Zod, Joi, ou autre). Seuls quelques endpoints vérifient la présence de champs obligatoires. Cette lacune expose l'application à des injections de données malformées et à des violations de contraintes métier.

5. Analyse du Frontend (Pages & Composants)

Le frontend est la couche la plus aboutie visuellement, avec 60+ pages fonctionnelles et une interface cohérente basée sur `shadcn/ui`.

5.1 Espace Pro (24 pages)

Page documentée	Route réelle	Statut
dashboard	<code>/pro</code>	Implémenté - KPIs, graphiques, alertes
stock	<code>/pro/stock</code>	Implémenté - Inventaire complet avec CMUP
pos	<code>/pro/caisse</code>	Implémenté - POS complet (nom différent)
commandes	<code>/pro/commandes</code>	Implémenté - Commandes fournisseurs
patients	<code>/pro/patients</code>	Implémenté - Gestion patients + crédits
ordonnances	<code>/pro/ordonnances</code>	Implémenté - Upload, OCR, validation
rh	<code>/pro/personnel</code>	Implémenté - RH complet (nom différent)
finance	<code>/pro/finance</code>	Implémenté - Trésorerie, P&L, graphiques
garde	<code>/pro/garde</code>	Implémenté - Planning + rapports
conformite	<code>/pro/conformite</code>	Implémenté - Score 5 composantes
alertes	<code>/pro/alertes</code>	Implémenté - Alertes DPMED
analytics	<code>/pro/analytics</code>	Implémenté - Prédictions IA, radar

5.2 Pages Pro supplémentaires (non documentées)

Route	Fonctionnalité
/pro/ventes	Historique des ventes et réimpression de reçus
/pro/remboursables	Gestion tiers-payant et remboursements
/pro/communication	Notifications push + campagnes SMS
/pro/reseau	Réseau multi-officines + transferts de stock
/pro/audit	Logs d'audit avec filtres et export CSV
/pro/retours	Retours produits et SAV
/pro/abonnement	Gestion abonnement et factures
/pro/documents	Coffre-fort numérique
/pro/qualite	Pharmacovigilance + interactions DCI
/pro/fournisseurs	Gestion fournisseurs + évaluations
/pro/parametres	Paramètres pharmacie et utilisateurs
/pro/credits	Crédits patients et suivi paiements
/pro/stupefiants	Registre des stupéfiants

5.3 Espace Patient (16 pages)

L'espace patient est complet avec toutes les fonctionnalités documentées et des pages supplémentaires : connexion, inscription, recherche de médicaments, pharmacie la plus proche (carte Leaflet), garde, commande en ligne, suivi, ordonnances, notifications, fidélité, comparateur de prix, rappels, vérification de médicaments, vaccinations, profil et urgence. L'authentification patient utilise localStorage au lieu de JWT/NextAuth, ce qui est moins sécurisé.

5.4 Espace Institutionnel (9+ pages)

Les portails DPMED, SoBAPS et ABRP sont implémentés avec des tableaux de bord, des cartes de couverture, et des statistiques agrégées. Le portail grossiste est une route séparée (/grossistes/) plutôt qu'un sous-groupe de /institutions/. Le groupe de routes (network) documenté est absent, remplacé par /pro/reseau.

5.5 Problèmes frontend identifiés

Plusieurs problèmes de qualité frontend sont notables. Le TypeScript ignore les erreurs de build (ignoreBuildErrors: true dans next.config.ts). Le reactStrictMode est désactivé, ce qui peut masquer des bugs. Les types sont définis inline dans chaque page plutôt que dans un module partagé. La fonction formatFCFA() est redéfinie dans presque chaque page au lieu d'être centralisée. L'i18n est partiellement implémenté (next-intl installé mais tout le texte est en dur en français). Les icônes PWA (icon-192.png, icon-512.png) référencées dans manifest.json n'existent pas dans le répertoire public.

6. Sécurité et Multitenancy

6.1 État du multitenancy

La documentation exige que chaque requête Prisma filtre par `pharmacieId` et que le `TenantMiddleware` injecte `SET app.current_tenant`. En réalité, il n'y a pas de `TenantMiddleware` (pas de `NestJS`). Le filtrage par `pharmacieId` est appliqué de manière incohérente : certaines routes utilisent le `pharmacieId` du JWT (bonne pratique), d'autres acceptent un `pharmacieId` en paramètre de requête (risque de cross-tenant), et certaines n'appliquent aucun filtre. Le Row Level Security (RLS) PostgreSQL mentionné dans la documentation n'est pas activé.

6.2 Authentification

La documentation spécifie Supabase Auth + JWT custom (15 min access / 7j refresh). Le code utilise NextAuth v4 avec CredentialsProvider (email/mot de passe) et stratégie JWT. Il n'y a pas de refresh token endpoint explicite. Le reset password stocke temporairement le token dans le champ `avatarUrl` de l'utilisateur, ce qui est un hack non sécurisé. Le rate limiting sur `/auth/login` (5 tentatives / 15 min / IP) documenté n'est pas implémenté.

6.3 Webhooks institutionnels

Les 4 webhooks institutionnels (DPMED, UbiPharm, Promopharma, SoBAPS) sont correctement sécurisés avec validation HMAC-SHA256 via le module `webhook-security.ts` qui utilise une comparaison `timing-safe`. C'est le point le plus sécurisé du codebase.

6.4 Éléments de sécurité manquants

Élément documenté	État	Sévérité
Signature RSA-256 pour alertes DPMED	Non implémentée (HMAC utilisé)	CRITIQUE
mTLS pour connexions institutions	Non implémenté	CRITIQUE
IP Whitelist sur webhooks	Non implémentée	HAUTE
Clés API chiffrées en base	Non implémenté	HAUTE
Logs d'audit sur actions sensibles	Partiel - table AuditLog existe mais pas systématique	MOYENNE
Rate limiting sur <code>/auth/login</code>	Non implémenté	HAUTE
Pseudonymisation signalements EI	Non implémentée	HAUTE
RLS PostgreSQL	Non activé	HAUTE

7. Stack Technique : Écarts Documentés vs Réels

Technologie documentée	État réel	Sévérité
NestJS + Fastify (backend)	ABSENT - Route Handlers Next.js	CRITIQUE
Supabase Auth	NextAuth v4 CredentialsProvider	MOYENNE
Redis via Upstash	ABSENT - Pas de cache, pas de session store	HAUTE
BullMQ (files d'attente)	ABSENT - Traitement synchrone	HAUTE
Sentry (monitoring)	ABSENT	MOYENNE
Vercel Analytics	ABSENT	FAIBLE
OpenStreetMap + Leaflet.js	Mapbox (coûts, clé API requise)	FAIBLE
next-pwa	Service Worker manuel (sw.js)	FAIBLE
SQLite offline (Prisma)	ABSENT - Pas de DB locale	CRITIQUE
Firebase FCM (push)	ABSENT - Pas de push notification	HAUTE
AfricasTalking (SMS)	ABSENT - Campagnes simulées	HAUTE
Fedapay (paiements)	Simulé - Appels API commentés	HAUTE
Resend (email)	ABSENT	MOYENNE

7.1 Dépendances non documentées

Le codebase inclut plusieurs dépendances non prévues dans la documentation : zustand (state management client), @tanstack/react-query et react-table, recharts (graphiques), jspdf + xlsx (export), sharp (images), qrcode, @mdxeditor/editor, framer-motion (animations), mapbox-gl + react-map-gl, cmdk (palette de commandes), et z-ai-web-dev-sdk. Ces ajouts enrichissent l'application mais n'étaient pas spécifiés, ce qui suggère un développement organique plutôt que guidé par les spécifications.

8. Modules : État d'Avancement

Cette section évalue chaque module en comparant la documentation, le schéma Prisma, les routes API et les pages frontend.

Module	Prisma	API	Frontend	Global
M01 - Stock	Complet	Complet	Complet	90%
M02 - POS/Ventes	Partiel (pas de synchedAt)	Partiel (pas de /ventes/sync)	Complet	75%
M03 - Commandes	Complet	Complet	Complet	90%
M04 - Fournisseurs	Complet	Complet	Complet	90%
M05 - Patients	Complet	Complet	Complet	90%
M06 - Ordonnances	Complet	Complet	Complet	90%
M07 - Personnel (RH)	Complet	Complet	Complet	90%

Module	Prisma	API	Frontend	Global
M08 - Finance	Complet	Complet	Complet	85%
M09 - Garde	Complet	Complet	Complet	90%
M10 - Remboursables	Complet	Complet	Complet	85%
M11 - Retours/Destructions	Complet	Complet	Complet	85%
M12 - Communication	Complet	Partiel (pas de push/SMS réel)	Complet	65%
M13 - Documents	Complet	Complet	Complet	85%
M14 - Dashboard	Complet	Complet	Complet	90%
M15 - Analytics IA	Partiel (pas de DomaineIA)	Partiel	Complet	70%
M16 - Pharmacovigilance	Complet	Complet	Complet	85%
M17 - Grossistes	Complet	Complet	Complet	80%
M18 - Alertes DPMED	Complet	Complet	Complet	75%
M19 - Conformité	Complet	Complet	Complet	85%

Les modules M01 à M11 sont les plus aboutis car ils correspondent aux phases 0-3 du développement. Les modules institutionnels (M16-M19) ont un schéma Prisma complet et des pages frontend fonctionnelles, mais manquent d'intégrations réelles avec les partenaires (pas de connexion réelle aux API des grossistes, pas de push FCM, pas de SMS AfricasTalking). Le mode offline (critique pour le Bénin) est le point faible majeur : pas de SQLite local, pas de synchronisation, pas de champ synchedAt.

9. Risques Critiques Identifiés

9.1 Risques de sécurité (Sévérité CRITIQUE)

L'absence quasi-totale d'authentification sur les routes API est le risque le plus grave. Sur 112 routes, seules 7 vérifient l'authentification. Le middleware vérifie la présence du cookie mais pas la validité de la signature JWT. Le pharmacieId est souvent pris du côté client (query parameter) plutôt qu'extrait du JWT, permettant un accès cross-tenant. Des identifiants de base de données sont en clair dans le code source (scripts/update-pharmacies-osm.ts). Le reset password utilise le champ avatarUrl comme stockage temporaire de token.

9.2 Mode offline inexistant (Sévérité CRITIQUE)

La documentation stipule que le POS et le Stock doivent fonctionner impérativement hors connexion, avec SQLite local, synchedAt pour tracer les ventes offline, et un endpoint POST /ventes/sync. Aucune de ces fonctionnalités n'est implémentée. C'est un risque majeur pour le marché béninois où la connexion internet est instable et les coupures électriques fréquentes. La promesse fondamentale du produit - un POS qui survit aux coupures - n'est pas tenue.

9.3 Intégrations externes simulées (Sévérité HAUTE)

Fedapay (paiements), AfricasTalking (SMS), Firebase FCM (push), et les API grossistes (UbiPharm, Promopharma) sont tous simulés ou absents. Les appels API Fedapay sont commentés. Les campagnes SMS n'envoient rien. Les push notifications n'existent pas. Les alertes DPMED sont stockées en base mais jamais diffusées par push ou SMS. Le flux critique documenté (diffusion en moins de 2 minutes) ne peut pas fonctionner sans BullMQ, Redis et Firebase FCM.

9.4 Absence de tests (Sévérité HAUTE)

Aucun framework de test n'est configuré. Pas de Jest, Vitest, Playwright ou Cypress. Pas de tests unitaires, d'intégration ou end-to-end. Pour une plateforme qui gère des données de santé et des transactions financières, c'est un risque inacceptable. Le TypeScript ignore les erreurs de build (`ignoreBuildErrors: true`) et l'ESLint a toutes ses règles désactivées, ce qui signifie que le code peut contenir des bugs non détectés.

10. Points Forts du Codebase

Malgré les écarts identifiés, le codebase présente des qualités remarquables qui méritent d'être soulignées.

10.1 Couverture fonctionnelle exceptionnelle

Les 19 modules documentés sont tous représentés dans le schéma Prisma et ont des pages frontend fonctionnelles. Les 60+ pages ne sont pas des maquettes : chacune intègre des appels API réels, des états de chargement (Skeleton), de la pagination, et des formulaires fonctionnels. Le POS (caisse) gère la création de ventes, la sélection de lots, les paiements multiples et la génération de reçus PDF. Le tableau de bord pro affiche des KPIs en temps réel avec des graphiques recharts.

10.2 Schéma Prisma de qualité production

Le schéma avec 75 modèles et 31 enums est extrêmement complet. Le RBAC relationnel (Role/Permission/RolePermission) est plus sophistiqué que le simple enum documenté. Les modèles institutionnels (AlerteDPMED, MedicamentSurveillance, ScoreConformite) sont plus riches que prévu. Le seed script (1019 lignes) utilise des upserts idempotents et des données réelles de Parakou avec coordonnées OSM.

10.3 Design cohérent et professionnel

L'interface utilise shadcn/ui avec une palette de couleurs cohérente (vert #1D9E75 primaire, #085041 sombre, #EF9F27 accent). L'espace patient est mobile-first avec bottom navigation et animations framer-motion. L'espace pro est professionnel avec des tableaux de bord riches. L'espace institutionnel a des cartes de couverture interactives. Le PWA est configuré avec manifest.json et service worker.

10.4 Infrastructure de sécurité (en partie)

Les webhooks institutionnels sont correctement sécurisés avec HMAC-SHA256 et comparaison timing-safe. Le module RBAC est complet avec 11 rôles et permissions par module/action. Le middleware NextAuth vérifie les sessions. L'architecture est prête pour une sécurisation complète - il manque principalement l'application systématique des guards existants.

11. Recommandations Prioritaires

11.1 Priorité CRITIQUE - Sécurité

Il est impératif d'appliquer `requireAuth()` et `withAuth()` sur TOUTES les routes API protégées. Le `pharmacieId` doit être extrait du JWT, jamais du query parameter. Le helper `requirePharmacieAccess()` existe déjà mais n'est pas utilisé. Un rate limiting doit être ajouté sur `/auth/login`. Le token de reset password doit être stocké dans une table dédiée, pas dans `avatarUrl`. Les identifiants de base de données en clair dans le code doivent être supprimés immédiatement.

11.2 Priorité CRITIQUE - Mode offline

L'implémentation du mode offline est essentielle pour le marché cible. Il faut ajouter les champs `Vente.reference` et `Vente.synchedAt` au schéma Prisma, implémenter SQLite local via Prisma pour les tables critiques (médicaments, lots, patients en lecture ; ventes, lignes_vente, paiements en écriture), créer le endpoint POST `/ventes/sync` avec idempotence, et ajouter un indicateur visuel permanent de l'état de connexion dans le POS.

11.3 Priorité HAUTE - Intégrations externes

Les intégrations doivent passer de la simulation à la réalité. Il faut activer Fedapay (décommenter les appels API et tester avec les clés), intégrer AfricasTalking pour les SMS, configurer Firebase FCM pour les push notifications, mettre en place Redis + BullMQ pour les files d'attente (alertes DPMED, SMS, analytics), et connecter les API grossistes (UbiPharm, Promopharma) avec les webhooks existants.

11.4 Priorité HAUTE - Tests

Un framework de test doit être mis en place de toute urgence. Vitest est recommandé pour les tests unitaires et d'intégration, Playwright pour les tests end-to-end. Les tests doivent couvrir au minimum : l'authentification et le RBAC, le multitenancy (isolation `pharmacieId`), les flux critiques (ventes, alertes DPMED, commandes), et les webhooks institutionnels. Les configurations `ignoreBuildErrors` et `ESLint` permissif doivent être progressivement corrigés.

11.5 Priorité MOYENNE - Convergence documentation/code

La documentation et le code ont divergé significativement. Il faut soit mettre à jour la documentation pour refléter l'architecture actuelle (Next.js seul, pas de NestJS, pas de monorepo), soit planifier une migration vers l'architecture documentée. Les enums Prisma doivent être harmonisés (StatutVente, ModePaiement, StatutCommandePatient en français). Le schema Prisma doit être aligné avec les spécifications (ajout de DomaineIA, AnalyticsReport structuré). Enfin, les migrations versionnées (prisma migrate deploy) doivent remplacer db push pour la production.